

Documentação de Modelagem

Projeto de BI — Análise de Capacidade Operacional
Equipe de Suporte Técnico

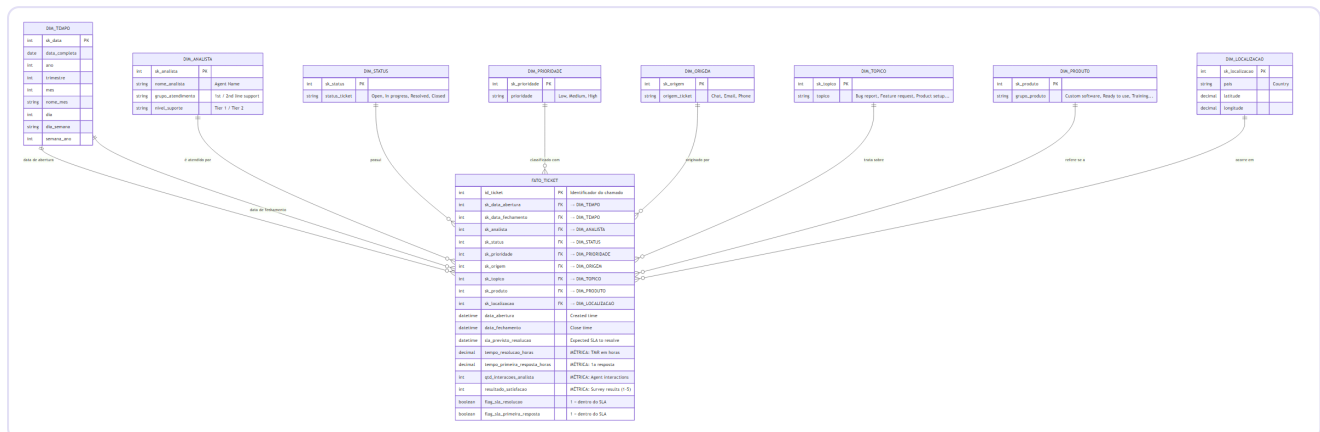
[MER](#) • [Diagrama de Classes](#) • [Diagrama de Casos de Uso](#)

Anderson Santos

github.com/AndersondSFonseca/Projeto-Capacidade-Operacional

1. Modelo de Dados (MER)

Modelo dimensional (esquema estrela) da base de tickets de suporte (~2.330 registros).



O que é

Modelo de dados desenhado como **esquema estrela**, padrão de Business Intelligence: no centro a tabela **Fato** (os tickets e suas métricas) e ao redor as **Dimensões** (atributos descritivos usados para filtrar e agrupar as análises).

Componentes

- **FATO_TICKET** — 1 linha = 1 ticket. Guarda as métricas (tempo de resolução, SLA, satisfação, interações) e as chaves para as dimensões.
- **Dimensões** — Tempo (abertura/fechamento), Analista, Prioridade, Status, Origem, Tópico, Produto e Localização.

Cardinalidade

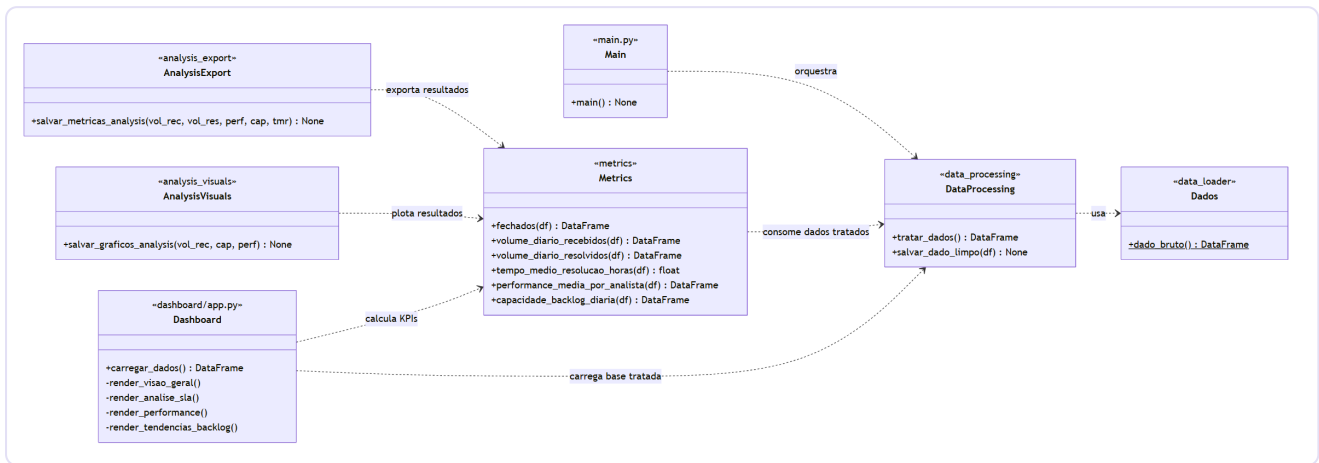
Todos os relacionamentos são **1:N (um-para-muitos)**: cada dimensão descreve muitos tickets; cada ticket aponta para uma única ocorrência de cada dimensão.

Como apresentar

"Este é o modelo de dados em esquema estrela. No centro a tabela Fato, com um registro por ticket e todas as métricas — tempo de resolução, SLA e satisfação. Ao redor, as dimensões, que são os filtros do dashboard: analista, prioridade, tempo, canal, tópico e localização. A relação é sempre um-para-muitos. É esse modelo que permite cruzar os dados."

2. Diagrama de Classes

Arquitetura do código-fonte em Python, organizada em camadas (pipeline de dados).



O que é

Representa os **módulos do código**, seus métodos e as **dependências** entre eles. Mostra que o projeto é uma solução organizada, com separação de responsabilidades.

O fluxo do dado (pipeline)

- **Dados (data_loader)** — lê o CSV bruto.
- **DataProcessing** — ETL: traduz colunas, limpa, converte datas, calcula tempos.
- **Metrics** — volume recebido/resolvido, tempo médio, performance, backlog.
- **AnalysisExport / AnalysisVisuals** — exportam CSVs e gráficos.
- **Dashboard (app.py)** — apresenta tudo no painel Streamlit.
- **Main** — orquestra o pipeline de tratamento.

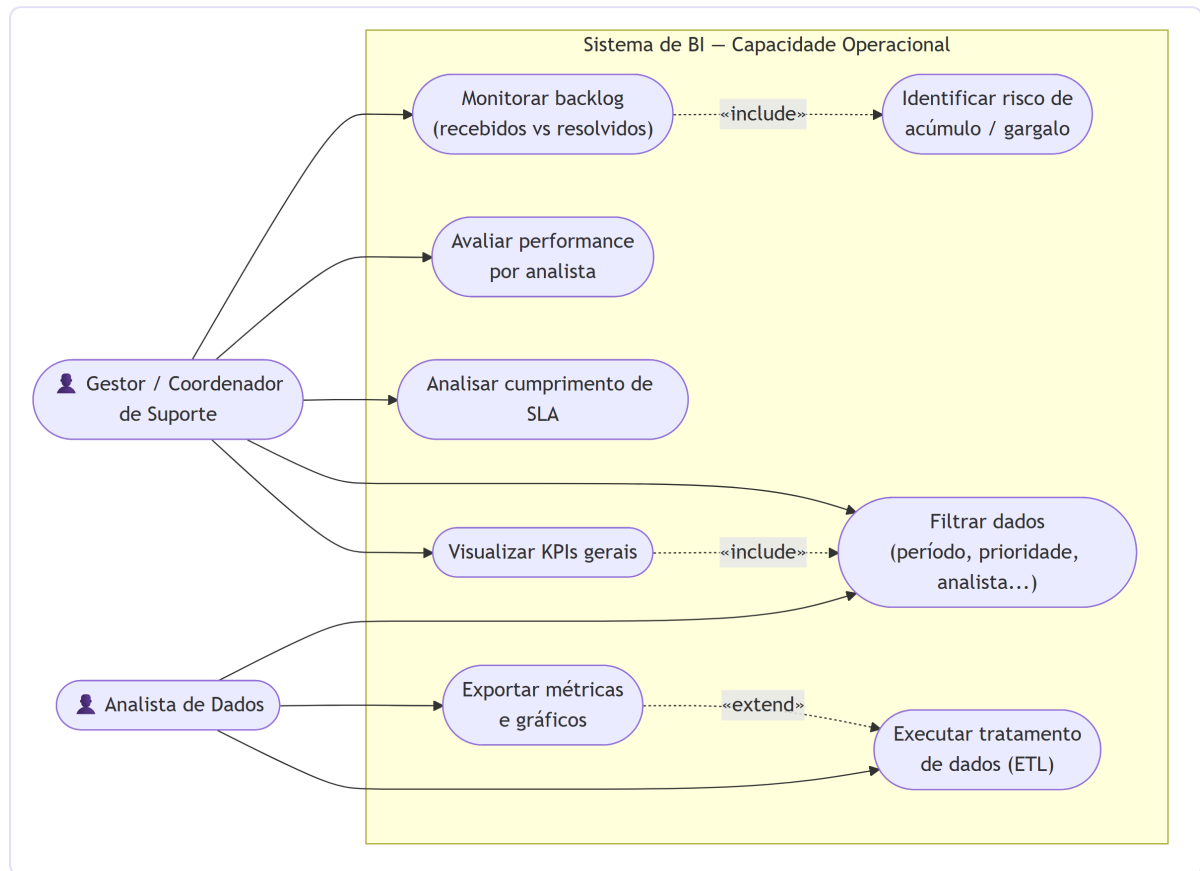
As setas tracejadas significam "usa / depende de".

Como apresentar

"Aqui está a arquitetura do código em camadas. O dado entra pelo data_loader, passa pelo tratamento no data_processing, vira métrica no metrics e é apresentado no dashboard. Cada módulo tem uma responsabilidade única — isso deixa o projeto fácil de manter e testar."

3. Diagrama de Casos de Uso

Visão de negócio: quem usa o sistema e o que cada ator consegue fazer.



O que é

Mostra os **atores** e os **casos de uso** dentro da fronteira do sistema. Responde "para que serve e para quem", do ponto de vista do usuário.

Componentes

Ator	O que faz
Gestor / Coordenador de Suporte	Visualiza KPIs, analisa SLA, avalia performance dos analistas, monitora backlog e identifica risco de gargalo.
Analista de Dados	Executa o tratamento de dados (ETL) e exporta métricas e gráficos.

«**include**»: um caso sempre usa outro (monitorar backlog inclui avaliar o risco).

«**extend**»: um caso opcional estende outro (exportar estende o ETL).

Como apresentar

"Este diagrama mostra quem usa o sistema. O Gestor usa o dashboard para ver KPIs, acompanhar SLA, avaliar a performance e monitorar o backlog. O Analista de Dados cuida do pipeline: roda o ETL e exporta as métricas. É a visão de negócio do projeto."

